MONTE-CARLO TOPK: FAST TOP-K SELECTION FOR VECTOR SEARCH SCORES

A PREPRINT

Jian Qin

June 2026

ABSTRACT

Flat vector search computes one score per database vector and then selects the best K scores. On modern GPU baselines, this final selection can be a large part of the search time. Monte-Carlo TopK reduces this cost with a simple post-score selector. For each query, it samples a small set of scores, chooses a sampled threshold, filters the full score row once, and runs exact top-K only on the surviving candidates. In exact mode, rows with too few or too many candidates fall back to exact selection; an optional capped mode trades exact agreement for bounded work. In FAISS GPU flat-search tests on four ANN10M squared-L2 benchmarks, Monte-Carlo TopK gives a 15.0x selector speedup and a 2.62–2.83x end-to-end speedup, with zero fallbacks across five sample seeds. On SIFT1M, a no-fallback setting gives a 10.0x selector speedup and a 2.72x end-to-end speedup.

Keywords vector search · nearest-neighbor search · top-K selection · GPU search · FAISS

1 Introduction

Nearest-neighbor search returns the vectors with the smallest distances or the largest similarities to a query. In flat search, the system computes N scores for each query and then selects the best K entries. FAISS showed that this K-selection step can be a substantial part of the runtime after the distance kernel is optimized [5, 12].

This paper studies a narrow but common setting: after a flat-search score row is computed, the remaining task is to select its top-K entries. Monte-Carlo TopK is a simple selector for this setting. It samples a small set of scores, uses a sampled order statistic as a threshold, scans the full score row once, and runs exact top-K on the candidates below the threshold. The method does not reduce the full distance computation, change the index, or change the vector representation; Section 3 defines this selector and its exact-mode checks. It is useful when exact selection is expensive relative to the rest of the pipeline.

The main practical issue is the threshold. If it keeps fewer than K candidates, the row cannot return K results. If it keeps too many candidates, the candidate buffer overflows. The exact mode handles both cases by falling back to full-row exact selection. A separate capped mode avoids overflow fallbacks by keeping only a bounded number of candidates, but this mode is approximate. The experiments therefore report speed, candidate-count tails, fallback counts, returned-score agreement, and identifier overlap separately. This separation matters because equal scores at the K-th boundary can make two valid exact selectors return different identifiers.

The FAISS GPU results show that the approach can give a large selector gain in flat squared-L2 search. On four ANN10M datasets, Monte-Carlo TopK gives a 15.0x selector speedup and a 2.62–2.83x end-to-end speedup with zero fallbacks across five sample seeds. On SIFT1M, a no-fallback setting gives a 10.0x selector speedup and a 2.72x end-to-end speedup. These gains are not from reducing distance work; they come from reducing the cost of the final selection.

The contributions are:

1. A post-score top-K selector that uses a sampled threshold, a candidate buffer, and exact top-K selection over candidates.

2. An exact-mode contract with explicit underflow and overflow fallbacks, plus a separate approximate capped mode.
3. FAISS CPU and GPU experiments on SIFT1M and four ANN10M datasets, including speed breakdowns, candidate tails, fallback counts, and tie-aware correctness checks.

2 Related Work

Large nearest-neighbor systems usually get faster by changing candidate generation, compression, or the index. Inverted files and product quantization use coarse partitions and compressed distances [11, 19]. Graph indexes such as HNSW and NSG change the traversal path [8, 13]. Surveys group ANN systems into graph, quantization, partitioning, and hashing methods [2]. Monte-Carlo TopK is complementary to these methods: it keeps score computation unchanged and changes only the final selection step.

FAISS is the closest system baseline. Its GPU design showed that K-selection can take a large share of runtime once distance computation is fast [12]. FAISS provides GPU and CPU search tools [5, 14]. This paper compares with FAISS exact flat-search baselines and does not claim to replace indexing.

Top-K selection has its own literature. Classic exact order-statistic algorithms define the baseline problem [7]. GPU-specific work includes randomized direct selection [15], radix and bucket selection [1], k-NN-motivated selection structures [20], bitonic top-K processing [18], SampleSelect-style selection through sampling and bucket counting [16, 17], and delegate-based GPU top-K filtering [9]. Monte-Carlo TopK also uses sampling and filtering, but its scope is narrower: vector-search score rows, returned K-th scores, buffer caps, fallback counts, and FAISS comparisons. The main exact GPU comparison is FAISS `runBlockSelect`.

Information retrieval and database systems also use thresholds [3, 4, 6]. Those methods rely on score bounds, sorted access, or external-sort state. In flat dense-vector search, an unsampled score is unknown until its distance is computed. Monte-Carlo TopK still computes the full score row in flat search.

3 Method

Let $X = \{x_i\}_{i=1}^{N}$ be the database, q a query, and $s_i = d(q, x_i)$ a score where smaller is better. Exact flat search computes all N scores and returns the indices of the K smallest values. Monte-Carlo TopK keeps the same score matrix but changes the selection step. Figure 1 summarizes the data flow.

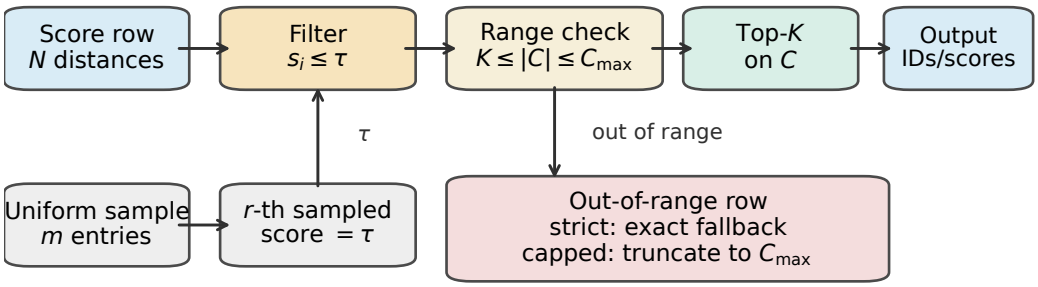


Figure 1: Monte-Carlo TopK is a post-score selector. It samples a threshold, filters the full score row into a bounded candidate buffer, and runs exact top-K on the candidates when the count is in range. Out-of-range rows either fall back to exact selection or use approximate overflow capping.

3.1 Algorithm

For each query, the algorithm has four stages:

1. Select a sample $S \subset \{1, \ldots, N\}$ of size m.

2. Compute or gather sample scores $\{s_i : i \in S\}$ and choose the r-th smallest sample score as the sampled threshold τ.
3. Scan the full score row and form the candidate set $C = \{i : s_i \le \tau\}$.
4. If $K \le |C| \le C_{\max}$, where $C_{\max}$ is the candidate buffer cap, run exact top-K on $\{s_i : i \in C\}$. If $|C| < K$, fall back to exact top-K over all N scores for that row. If $|C| > C_{\max}$, either fall back to exact selection or use capped mode, which keeps at most $C_{\max}$ candidates and is approximate.

The threshold rank r controls the candidate count. With a uniform sample and no ties, the threshold quantile is about r/m, so the expected candidate count is about Nr/m. Useful settings keep this value above K and below the cap. Larger r reduces underflow risk but keeps more candidates. A larger cap reduces overflow risk but uses more memory.

Candidate-buffer memory is straightforward. For batch size Q_{batch}, candidate cap $C_{\max}$, 32-bit scores, and FAISS 64-bit IDs, score/id storage is about

$$Q_{\text{batch}} C_{\max} \left(\text{sizeof}(\text{float}) + \text{sizeof}(\text{idx_t})\right) \approx 12 Q_{\text{batch}} C_{\max} \text{ bytes.}$$

For GPU batch size 32, this is about 3.0 MiB at cap 8192, 6.0 MiB at 16384, and 18.3 MiB at 50000, before small counters and output buffers. The cap affects memory, latency, and accuracy.

For tuning, treat the sampled threshold as a random order statistic. With a sample of size m drawn without replacement from N unique ranks, the rank R_r of the r-th sample item has

$$\mathbb{E}[R_r] = \frac{r(N+1)}{m+1} \quad \text{and} \quad \text{Var}(R_r) = \frac{r(m-r+1)(N+1)(N-m)}{(m+1)^2(m+2)}.$$

The mean alone is insufficient. Underflow means $R_r < K$, or at least r samples landing among the first $K-1$ population ranks:

$$\Pr[c_q < K] = \Pr[\text{Hypergeom}(N, K-1, m) \ge r].$$

Overflow means $R_r > C_{\max}$, or fewer than r samples landing among the first $C_{\max}$ population ranks:

$$\Pr[c_q > C_{\max}] = \Pr[\text{Hypergeom}(N, C_{\max}, m) < r].$$

These equations give a starting rule from N and K. Choose a candidate multiplier $\alpha = \mathbb{E}[c]/K$. In exact mode, α should usually be several times larger than one. For sample size m, set

$$r \approx \left\lceil \alpha K \frac{m+1}{N+1} \right\rceil \approx \left\lceil \alpha K \frac{m}{N} \right\rceil.$$

If this gives a very small r, use a larger α or m. For a batch with Q rows, choose r so that $Q \Pr[c_q < K] \ll 1$. Choose $C_{\max}$ from a high quantile of R_r under the memory budget, then measure p99 and maximum candidates in a tuning run. This is a starting range, not a guarantee. Score ties, finite-precision distances, and query difficulty can change the tails.

The SIFT1M measurements follow this rule. With $N = 10^6$, $K = 64$, and $m = 8192$, $r = 4$ gives about 488 expected candidates and predicts about 18 underflows over 9984 queries. The measured run has 18 fallback rows. Raising r to 8 roughly doubles the candidates and removes fallbacks. For $K = 1024$, the same sample size needs `sample_k=32`. The experiments report underflow, overflow, and fallback counters with every result.

3.2 Result Checks

Let γ_K be the K-th smallest full-row score. If $K \le |C| \le C_{\max}$, then $\tau \ge \gamma_K$. Exact top-K on C returns the same K-th score, γ_K. If there are no ties at γ_K, the returned IDs also match full-row exact top-K. With ties, several ID sets can be valid, so ID overlap can be below 1.0 even when the score is correct.

Exact mode falls back when fewer than K scores pass, or when more than $C_{\max}$ scores pass. Results must report underflow, overflow, fallback count, ID overlap, and the returned K-th score. Tie-aware overlap explains low ID overlap when many vectors share the exact K-th score.

Overflow capping is different from threshold filtering. If $|C| > C_{\max}$, capped mode keeps only $C_{\max}$ candidates and does not fall back. This is faster, but it can return a worse K-th score. It must report overlap against exact top-K.

3.3 Cost Compared With Exact Selection

For Q queries, dimension D, database size N, sample size m, output size K, and candidate count $c_q = |C_q|$, full distance computation costs

$$T_{\text{dist}} = \Theta(QND).$$

Exact flat search then runs a full-row selector. A heap-style selector costs $O(N \log K)$ per query. GPU selectors can be faster, but they still read the full score row. Let the exact selection cost be

$$T_{\text{exact-select}} = \Theta(Q\, S(N, K)),$$

where $S(N, K)$ is the cost of exact top-K over N scores for a given code path.

Monte-Carlo TopK adds sampling, one full filter scan, and a smaller candidate selector:

$$T_{\text{MC-select}} = \Theta(Q\, S(m, r)) + \Theta(QN) + \sum_{q=1}^{Q} \Theta(S(c_q, K)) + T_{\text{fallback}}.$$

If sample distances are computed separately, the method also pays $\Theta(QmD)$. If the full score row is already stored, this becomes a gather over m scores. In flat search, $\Theta(QND)$ remains unchanged:

$$T_{\text{MC-flat}} = \Theta(QND) + \Theta(QmD) + T_{\text{MC-select}}.$$

This is not a claim of lower distance cost. A speedup needs

$$S(N, K) \gg S(m, r) + N + \mathbb{E}[S(c_q, K)],$$

with small fallback cost. If distance computation takes most of the time, end-to-end speedup is limited. With overflow capping, $S(c_q, K)$ is replaced by $S(\min(c_q, C_{\max}), K)$, but rows with $c_q > C_{\max}$ become approximate.

3.4 Fallback Avoidance

Fallbacks are rare but expensive. Underflow means $c_q < K$; overflow means $c_q > C_{\max}$. The experiments use three safeguards:

1. Choose the threshold rank so the expected candidate count is several times larger than K, not just equal to K.
2. Set the candidate cap using high-percentile candidate counts from a tuning run, not only the mean.
3. Report fallback counts for every speed result, because a few slow fallback rows can take most of the total selector time.

Table 3 shows the effect. A setting with 18 fallback rows is slower than exact top-K. A safer threshold with zero fallbacks gives nearly 10x selector speedup.

4 Experiments

4.1 Setup

The experiments use SIFT1M [10, 11] and four public ANN10M xbin subsets: BIGANN, DEEP, Microsoft Turing-ANNS, and Microsoft SPACEV. Unless stated otherwise, the task is flat squared-L2 search followed by top-K selection. The GPU baseline uses FAISS v1.14.3 [14]: cuBLAS computes squared-L2 scores, and `runBlockSelect` performs exact top-K selection. The CPU comparison uses FAISS `IndexFlatL2::search` and `knn_L2sqr`.

The main timing metric is end-to-end speedup: exact flat-search time divided by Monte-Carlo TopK time, including sample scoring, filtering, candidate top-K, and fallback. Selector speedup compares only exact top-K selection with Monte-Carlo TopK selection. Correctness is reported in three ways. ID overlap is the overlap with the exact top-K identifiers. Returned-score agreement checks whether the returned K-th score matches exact selection. Tie-aware overlap also accepts identifiers tied with the exact K-th score. Tables abbreviate returned-score agreement and tie-aware overlap minima as "score/tie min."

These metrics separate correctness from tie-breaking and parameter safety. A low ID overlap indicates an error only if the returned score boundary changes or if tie-aware overlap also falls. If returned-score agreement and tie-aware overlap are both 1.0, the row has an exact score boundary and the ID difference comes from arbitrary ordering among equal-score boundary items. We therefore also report boundary-tie counts. Separately, underflow, overflow, and fallback counts show whether a parameter setting is safe, while candidate p99/max and selector p99 show whether the cap and tail latency are well controlled.

The benchmark flags map to the notation in Section 3 as `sample_n`=m, `sample_k`=r, and `candidate_cap`=$C_{\max}$.

4.2 FAISS GPU on ANN10M

Table 1: FAISS GPU ANN10M flat-search benchmark on RTX 4090 over five sample seeds. All rows use 1024 queries, $K = 64$, `sample_n=32768`, `sample_k=4`, and `candidate_cap=8192`. Speedups and candidate means are mean/std across seeds; the p99 column gives the range across seeds.

Dataset	dtype	Exact	MC	Speedup	Select speedup	Cand. mean	Cand. p99	ID min	Score/tie min	Tie rows/max	Fallbacks
BigANN	uint8	1.0544	0.4024	2.620±0.002x	15.077±0.013x	1223±11	2862–3052	0.9844	1.0/1.0	14/3	0
Deep	float32	1.0081	0.3561	2.831±0.001x	15.073±0.009x	1225±7	2784–3116	0.3438	1.0/1.0	38/191	0
MSSPACEV	int8	1.0191	0.3672	2.775±0.001x	15.071±0.006x	1227±15	2930–3223	0.1250	1.0/1.0	68/1240	0
MSTuring	float32	1.0193	0.3676	2.773±0.001x	15.068±0.010x	1218±28	2844–3142	1.0000	1.0/1.0	31/3	0

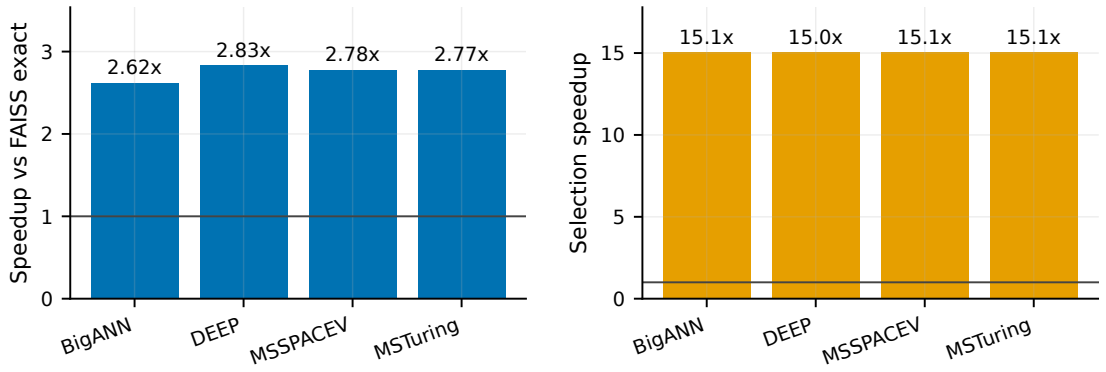


Figure 2: FAISS GPU ANN10M speedups. All four datasets have zero underflows, overflows, and fallbacks across five sample seeds.

Table 1 and Figure 2 show the main GPU result. Monte-Carlo TopK reduces FAISS top-K selection from about 0.700 s to 0.046 s, a 15.0x selector speedup. End-to-end flat-search speedup is 2.62x–2.83x. All rows have zero underflows, zero overflows, and zero fallbacks across five sample seeds.

The low worst-row ID overlap on Deep and MSSPACEV is caused by equal-score boundary ties. In every row, the returned K-th score matches exact selection, and the tie-aware overlap minimum is 1.0. The largest boundary tie has 191 identifiers on Deep and 1240 on MSSPACEV. The observed candidate means (1218–1227) match the order-statistic estimate $10^7 \cdot 4/32768 = 1220.7$, while the p99 range is wider than an iid model would predict.

Table 2: GPU cap and parameter sweep on the tie-heavy ANN10M datasets (one seed). Strict mode falls back on overflow; capped mode is approximate and keeps at most $C_{\max}$ candidates. Capped rows use small caps to expose the quality/latency trade-off. The $m/r/C_{\max}$ column gives sample size, sample rank, and candidate cap. All rows use 1024 queries, $K = 64$, and batch size 32.

Dataset	Mode	$m/r/C_{\max}$	Speedup	Select speedup	Cand. p99/max	ID min	Score/tie min	Overflow/fallback	Select p99
Deep	strict	32768/4/4096	2.834x	15.14x	2949/4009	0.4219	1.0/1.0	0/0	1.45 ms
Deep	strict	32768/4/8192	2.832x	15.08x	2949/4009	0.4219	1.0/1.0	0/0	1.45 ms
Deep	strict	65536/8/8192	2.809x	14.36x	2506/3346	0.4219	1.0/1.0	0/0	1.53 ms
Deep	capped	32768/4/2048	2.835x	15.18x	2949/4009	0.4375	0.0/0.5	101/0	1.45 ms
Deep	capped	32768/4/4096	2.835x	15.14x	2949/4009	0.4375	1.0/1.0	0/0	1.45 ms
MSSPACEV	strict	32768/4/4096	0.289x	0.22x	3223/4354	0.2188	1.0/1.0	1/1	2182.13 ms
MSSPACEV	strict	32768/4/8192	2.772x	15.06x	3223/4354	0.0625	1.0/1.0	0/0	1.45 ms
MSSPACEV	strict	65536/8/8192	2.750x	14.34x	2467/3121	0.0625	1.0/1.0	0/0	1.53 ms
MSSPACEV	capped	32768/4/2048	2.779x	15.19x	3223/4354	0.1719	0.0/0.5	115/0	1.44 ms
MSSPACEV	capped	32768/4/4096	2.777x	15.13x	3223/4354	0.1094	0.0/0.9844	1/0	1.45 ms

Table 2 separates parameter tuning from tie handling. Larger samples reduce the candidate tail: Deep p99/max drops from 2949/4009 to 2506/3346, and MSSPACEV drops from 3223/4354 to 2467/3121. The raw ID minimum does not become 1.0 because FAISS exact selection and the candidate selector may choose different valid identifiers from the same boundary-score tie set. Achieving raw ID overlap 1.0 on these rows would require a stable score–ID tie-breaker in both selectors, not a larger cap. The sweep also explains the GPU cap choice. MSSPACEV with cap 4096 has one overflow fallback and the selector p99 rises to 2.18 s; cap 8192 removes the fallback. The capped rows use cap 2048

and 4096 as stress points below or near the observed candidate maximum. Deep at cap 4096 has no overflow, so capped and strict mode agree. MSSPACEV at cap 4096 still truncates one overflow row and has returned-score agreement minimum 0.0. A cap of 8192 has no overflow in this sweep, so capped mode is equivalent to the strict row and is not listed separately.

4.3 SIFT1M Parameter Behavior

Table 3: FAISS SIFT1M flat-search rows on RTX 4090 using exact fallback. Candidate statistics are p50/p95/p99/max. All rows have returned-score agreement and tie-aware overlap 1.0.

Q	K	sample_k	Speedup	Select speedup	ID min	Candidates	Under/over/fallback	Select p99	Tie rows/max
1024	64	4	2.73x	10.25x	0.9844	446/954/1166/1579	0/0/0	0.22 ms	41/2
9984	64	4	0.165x	0.122x	0.9844	444/956/1216/2064	18/0/18	316 ms	455/3
9984	64	8	2.72x	10.02x	0.9844	934/1595/1927/3203	0/0/0	0.23 ms	455/3
1024	1024	32	3.19x	22.69x	0.9990	3813/5066/5573/6423	0/0/0	0.32 ms	148/3

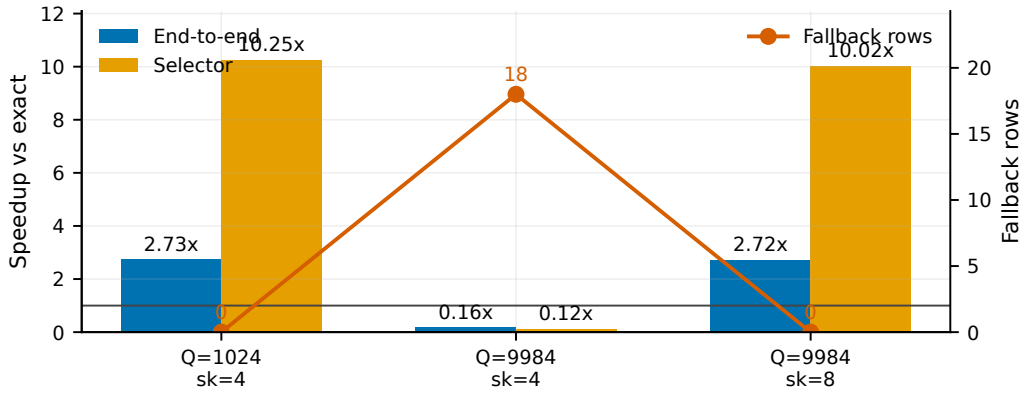


Figure 3: SIFT1M parameter behavior. The full-query `sample_k=4` row triggers 18 underflow fallbacks and becomes slower than exact selection, while `sample_k=8` avoids fallback and recovers the expected selector gain.

Table 3 and Figure 3 show that the threshold rank must be chosen for the tail, not only for the mean. With `sample_k=8`, the full-query run has zero fallbacks, a 10.02x selector speedup, and a 2.72x end-to-end speedup. With `sample_k=4`, 18 underflow rows fall back to full-row exact selection, and the run becomes slower than exact top-K. The practical tuning rule is therefore to avoid fallback first, then reduce the candidate count.

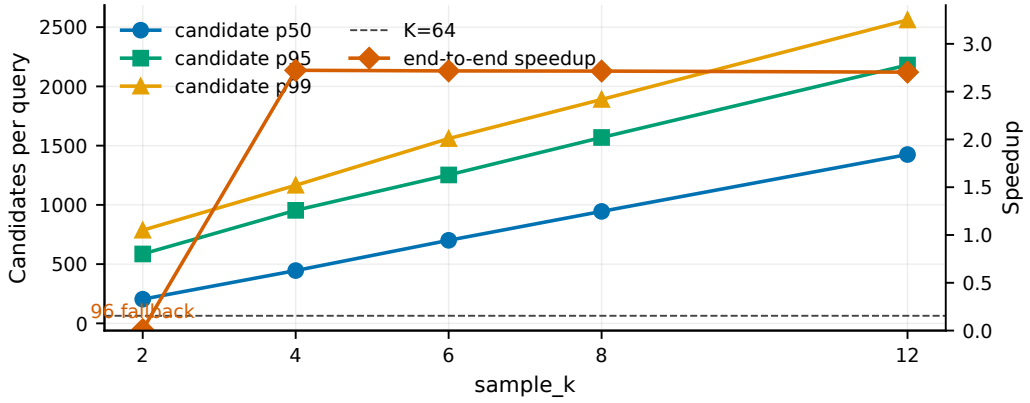


Figure 4: SIFT1M `sample_k` sensitivity for 1024 queries and $K = 64$. `sample_k=2` keeps too few candidates and triggers 96 fallbacks. Larger values increase candidate quantiles but keep returned-score agreement and tie-aware overlap 1.0.

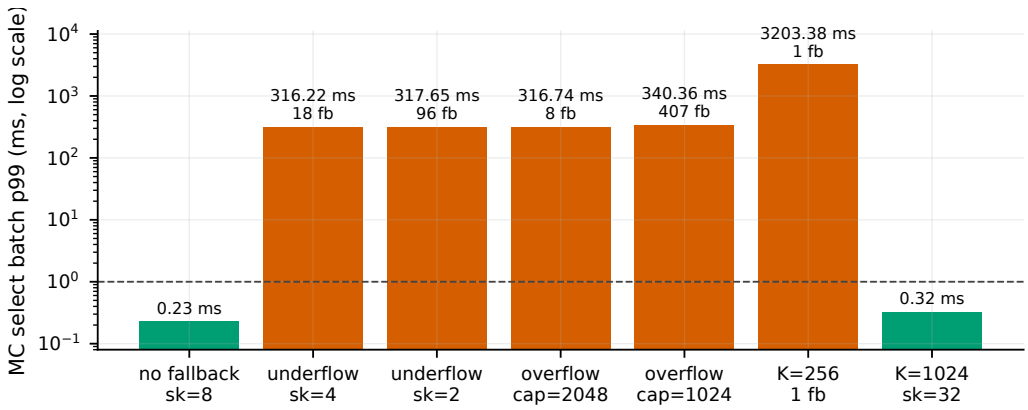


Figure 5: Tail latency is set by fallback rows. Rows without fallback have sub-millisecond MC select p99, while underflow or overflow fallback pushes p99 to hundreds of milliseconds; one $K = 256$ underflow row raises p99 above 3 s.

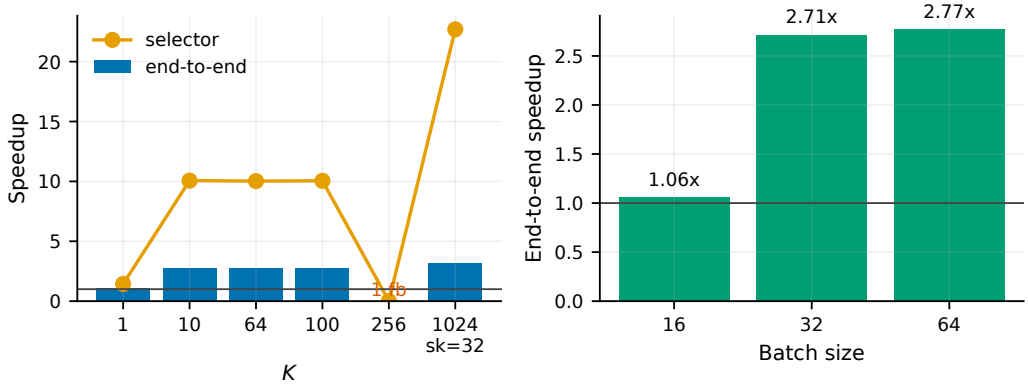


Figure 6: Extra SIFT1M sensitivity. $K = 1$ leaves little selection time to save; $K = 1024$ requires a larger threshold rank (`sample_k=32`) to avoid underflow and then recovers 3.19x end-to-end speedup. Batch-size results show that throughput and sample scoring can shift the end-to-end gain.

Figures 4, 5, and 6 show the same pattern in sweeps over `sample_k`, K, and batch size. Fallback rows dominate tail latency. In no-fallback rows, the selector p99 is below 0.4 ms in these SIFT1M runs.

4.4 CPU and Capped Mode

Table 4 shows the CPU result. Monte-Carlo TopK is 1.85–3.12x faster than FAISS `IndexFlatL2::search` and 2.00–3.29x faster than `knn_L2sqr`, with exact top-K agreement and zero fallbacks.

Table 4: FAISS CPU ANN10M comparison on an Intel Core i9-13900KF. All rows use 1024 queries, $K = 64$, `sample_n=32768`, `sample_k=4`, and `candidate_cap=8192`. Times are seconds.

Dataset	dtype	IndexFlatL2	`knn_L2sqr`	MC total	IndexFlat/MC	knn/MC	ID overlap
BigANN	uint8	21.773	22.992	6.985	3.12x	3.29x	1.000
Deep	float32	16.022	17.032	5.772	2.78x	2.95x	1.000
MSSPACEV	int8	11.692	12.269	5.985	1.95x	2.05x	1.000
MSTuring	float32	11.099	12.016	6.008	1.85x	2.00x	1.000

Table 5: FAISS overflow-capping stress test based on SIFT1M. Effective 10M rows repeat the SIFT1M score matrix; they are not physical 10M dataset runs. Candidates are mean/min/max.

Effective N	Cap	MC total	Phys. speedup	Select speedup	ID overlap mean/min	Candidates	Overflows	Fallbacks
1M	256	0.0348	2.92x	14.88x	0.6112/0.0625	490/66/1579	863	0
10M	2048	0.1222	2.92x	15.74x	0.5542/0.0938	4902/660/15790	935	0
10M	4096	0.1226	2.92x	15.63x	0.8540/0.2031	4902/660/15790	589	0
10M	8192	0.1231	2.91x	15.46x	0.9901/0.5313	4902/660/15790	106	0
10M	16384	0.1232	2.91x	15.33x	1.0000/0.9844	4902/660/15790	0	0

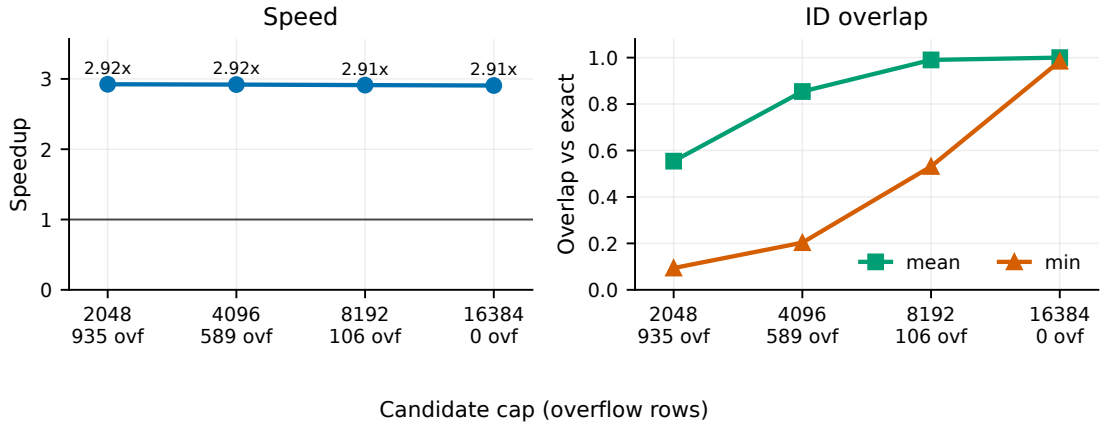


Figure 7: Overflow capping bounds selector work but loses overlap. The plotted rows use the effective 10M repeated-score stress case.

Table 5 and Figure 7 isolate the approximate capped mode. At effective $N = 10$M, cap 8192 keeps selector speedup near 15.5x and mean ID overlap at 0.9901, but worst-row overlap is only 0.5313. Cap 16384 removes overflows and raises worst-row overlap to 0.9844. These rows are stress tests for the selector, not physical 10M end-to-end retrieval results.

5 Limitations and Future Work

Monte-Carlo TopK speeds up selection; it does not replace indexing. In flat search, it still needs the full score row. Its end-to-end gain depends on how much time exact search spends in selection. A natural next step is to combine it with product quantization, coarse clustering, or graph search.

The current implementation also pays a high cost for fallback. In SIFT1M, 18 underflow fallback rows make the full run slower than exact top-K. A production implementation should use the same full-row selector as the exact baseline, or retry only the rows with too few or too many candidates.

The current experiments report returned score, ID overlap, equal-score ties, candidate tails, and tail latency. They still do not cover all K values, batch sizes, dimensions, data types, query shards, metrics, or indexed pipelines. The results are for squared-L2 flat search. Inner product and cosine search should be tested before making broader claims. A deployed system should choose the threshold rank from online candidate statistics or from a calibration pass, not only from a fixed table.

The implementation leaves several opportunities for optimization. It uses separate kernels for sampled threshold selection, filtering/counting, candidate compaction, and candidate top-K. A tighter implementation could fuse adjacent stages, use prefix-sum compaction where it reduces atomic contention, overlap sample scoring with full distance computation, and choose threshold ranks per row or per batch. These changes could reduce overhead and fallback risk; they do not change the main limitation that flat search still computes the full score row.

6 Conclusion

Monte-Carlo TopK is a post-score selector for vector search. It samples a threshold, filters the score row once, and runs exact top-K on a smaller candidate set. In FAISS GPU tests, it gives a 15.0x selector speedup and a 2.62–2.83x end-to-end speedup on four ANN10M datasets, with zero fallbacks across five sample seeds. The remaining ID differences are explained by equal-score boundary ties.

The main practical limit is tuning. The threshold must keep enough candidates for exact top-K while staying below the cap. The simple Nr/m rule sets the mean, but measured tails set the cap. The method is therefore best viewed as a selection primitive that can complement, rather than replace, indexed search.

Code Availability

The FAISS GPU implementation used for the reported benchmark results is available at `https://github.com/qinjian623/faiss`. The main selector code is under `faiss/gpu/utils/MonteCarloTopK.*`; the GPU tests and benchmark drivers are under `faiss/gpu/test/`.

References

[1] Tolu Alabi, Jeffrey D. Blanchard, Bradley Gordon, and Russel Steinbach. Fast K-selection algorithms for graphics processing units. *ACM Journal of Experimental Algorithmics*, 17, 2012. doi: 10.1145/2133803.2345676.

[2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020. doi: 10.1016/j.is.2019.02.006.

[3] Andrei Z. Broder, David Carmel, Michael Herscovici, Aya Soffer, and Jason Zien. Efficient query evaluation using a two-level retrieval process. In *Proceedings of the Twelfth International Conference on Information and Knowledge Management*, pages 426–434, 2003. doi: 10.1145/956863.956944.

[4] Yannis Chronis, Thanh Do, Goetz Graefe, and Keith Peters. External merge sort for top-K queries. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 2423–2437. ACM, 2020. doi: 10.1145/3318464.3389729.

[5] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library, 2024. Version 4 revised 2025-10-23.

[6] Ronald Fagin, Amnon Lotem, and Moni Naor. Optimal aggregation algorithms for middleware. *Journal of Computer and System Sciences*, 66(4):614–656, 2003. doi: 10.1016/S0022-0000(03)00026-6.

[7] Robert W. Floyd and Ronald L. Rivest. Algorithm 489: SELECT–for finding the ith smallest of n elements. *Communications of the ACM*, 18(3):173, 1975. doi: 10.1145/360680.360694.

[8] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proceedings of the VLDB Endowment*, 12(5):461–474, 2019. doi: 10.14778/3303753.3303754.

[9] Anil Gaihre, Da Zheng, Scott Weitze, Lingda Li, Shuaiwen Leon Song, Caiwen Ding, Xiaoye S. Li, and Hang Liu. Dr. Top-k: Delegate-centric Top-k on GPUs. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 2021. doi: 10.1145/3458817.3476141.

[10] IRISA TEXMEX Project. TEXMEX ANN_SIFT1M corpus. `http://corpus-texmex.irisa.fr/`, 2026. Accessed 2026-06-16.

[11] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. doi: 10.1109/TBDATA.2019.2921572.

[13] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

[14] Meta AI. Faiss: A library for efficient similarity search and clustering of dense vectors. `https://github.com/facebookresearch/faiss`, 2026. Release v1.14.3, commit 0ca9df4; accessed 2026-06-16.

[15] Laura Monroe, Joanne Wendelberger, and Sarah Michalak. Randomized selection on the GPU. In *Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics*, pages 89–98. ACM, 2011. doi: 10.1145/2018323.2018338.

[16] Tobias Ribizel and Hartwig Anzt. Approximate and exact selection on GPUs. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops*, pages 471–478, 2019. doi: 10.1109/IPDPSW.2019.00088.

[17] Tobias Ribizel and Hartwig Anzt. Parallel selection on GPUs. *Parallel Computing*, 91:102588, 2020. doi: 10.1016/j.parco.2019.102588.

[18] Anil Shanbhag, Holger Pirk, and Samuel Madden. Efficient top-K query processing on massively parallel hardware. In *Proceedings of the 2018 International Conference on Management of Data*, pages 1557–1570. ACM, 2018. doi: 10.1145/3183713.3183735.

[19] Josef Sivic and Andrew Zisserman. Video Google: A text retrieval approach to object matching in videos. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1470–1477, 2003. doi: 10.1109/ICCV.2003.1238663.

[20] Xiaoxin Tang, Zhiyi Huang, David M. Eyers, Steven Mills, and Minyi Guo. Efficient selection algorithm for fast k-NN search on GPUs. In *2015 IEEE International Parallel and Distributed Processing Symposium*, pages 397–406, 2015. doi: 10.1109/IPDPS.2015.115.